Group Name: *Grow A Garden Trading Hub*

Project Name: *LARP (Local AI Refinement Platform)*

Short Description:

LARP *(Local AI Refinement Platform)* is an offline AI tool that runs in your terminal using Ollama. It helps developers improve prompts, write technical specifications, plan software architecture, and create AI agent rules all locally on your computer, with no cloud data sharing.

Group Members:
Alcantara, Carlos S.
Enriquez, Joshua R.
Mendoza, Paulo C.
Piangco, Manuel Ziankyle E

Public GitHub repository: https://github.com/huwaffle/LARP-HACK

==========
Demo Video:
https://drive.google.com/file/d/1MrIvseOe2chXV79i2UDJnm-SI3tesWhD/view?usp=sharing

LinkedIn Video URL: *https://lnkd.in/p/g2AtZqwt*

What runs locally?

LARP *(Local AI Refinement Platform)* runs locally on your computer using Ollama, which allows AI models to process prompts and generate results without sending data to the cloud. Its features, such as prompt enhancement, project planning, and code-related tasks, work directly on your device.

What requires internet?

The internet is only needed to download Ollama, AI models, and required dependencies during setup. Once everything is installed, ***LARP*** can work offline without relying on cloud AI services.

==========
Models used:

- Primary / Default SLM: `gemma3:4b` (Google Gemma 3 4B parameters, instruction-tuned, 4-bit quantized via Ollama).
- Supported Local Models:
    - `llama3.2:3b` (Meta Llama 3.2 3B, low resource, fast execution)

    - `qwen2.5:7b` / `qwen2.5-coder:7b` (Alibaba Qwen 2.5, optimized for technical specifications and coding)
    - `mistral:7b` (Mistral AI 7B v0.3, balanced general reasoning)
- Embeddings & Vision: None. Pure local generative SLM text inference.

Technologies and frameworks:

- Language & Runtime: Python 3.10+ (Standard library: `urllib.request`, `json`, `threading`, `difflib`, `colorsys`, `ctypes`).
- Local Inference Engine: Ollama (runs locally on `http://127.0.0.1:11434`, utilizing a llama.cpp C/C++ backend with GPU/CPU acceleration).
- Terminal User Interface (TUI):
    - `prompt_toolkit`: Full-screen layout engine, custom buffer processor, keybinding event loop, and live autocomplete popup.
    - `rich`: Markdown parsing, terminal ANSI color engine, and inline visual diff renderer.
- System Utilities: `pyperclip` (operating system clipboard access).
- Packaging: `setuptools` / `pyproject.toml` (exposes console script entrypoint `larp = larp.app:main`).
- Database & Persistence: Local filesystem only (`larp_output/`, `.larp_theme`). Zero external databases.

APIs and cloud services:

- None (0 Cloud APIs / 0 External Endpoints).
- LARP has zero remote network dependencies. All inference calls are strictly routed to `http://127.0.0.1:11434` on localhost.

Existing code and assets:

- Open-Source Libraries: `prompt_toolkit`, `rich`, `pyperclip`, `setuptools`.
- Pre-Existing Codebases / Starter Templates: None. Core application architecture, terminal engine, and feature pipelines were built from scratch during the hackathon sprint.
- UI Design Paradigm: Inspired by developer CLI tools including OpenCode and Claude Code.

AI development tools:

- Devin, Antigravity IDE, Cursor (utilized for pair programming and code generation assistance during the hackathon sprint).